# Understanding the ALU (Arithmetic Logic Unit)

---

## Introduction

The **Arithmetic Logic Unit (ALU)** is a fundamental component of the CPU that performs all arithmetic and logical operations. It is a combinational digital circuit at the heart of all processors.

**Main functions of the ALU:**

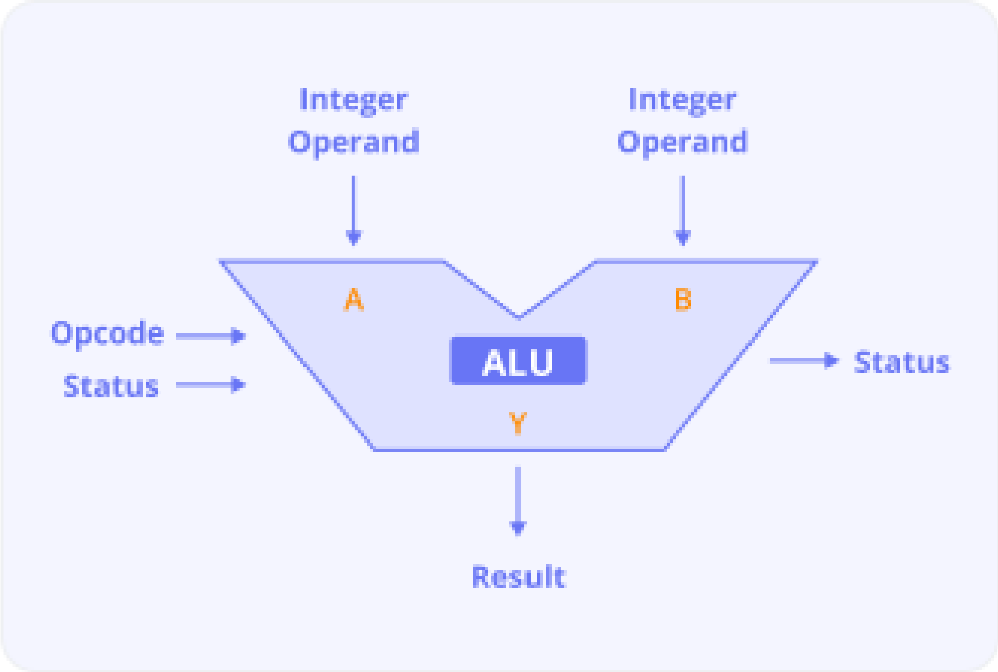
- Arithmetic operations: addition, subtraction, increment, decrement.
- Logical operations: AND, OR, NOT, XOR.
- Comparison operations: equal, greater than, less than.
- Shift operations: left shift, right shift.

## 1 Basic ALU Architecture

An ALU takes two binary inputs and performs an operation based on control signals.

### General Components

- **Input operands:** A and B
- **Control signals:** Select operation
- **Operation circuitry:** Arithmetic and logic sub-units
- **Flags:** Zero, Carry, Overflow, Sign
- **Output:** Result




# 2 ALU Operations

## 2.1 Arithmetic Operations

- Addition: $R = A + B$
- Subtraction: $R = A - B$
- Increment: $R = A + 1$
- Decrement: $R = A - 1$

## 2.2 Logical Operations

- AND: $R = A \wedge B$
- OR: $R = A \vee B$
- XOR: $R = A \oplus B$
- NOT: $R = \sim A$

## 2.3 Comparison Operations

- Equal: $(A = B)$
- Greater than: $(A > B)$
- Less than: $(A < B)$

### 2.4 Shift Operations

- Logical Left Shift: $A << 1$
- Logical Right Shift: $A >> 1$
- Arithmetic Right Shift: preserves sign bit

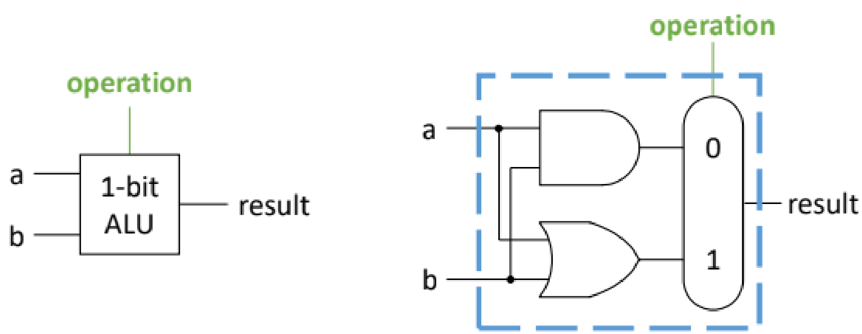
## 3 ALU Design Using Multiplexers

To create a compact ALU, we use multiplexers to select between operations.

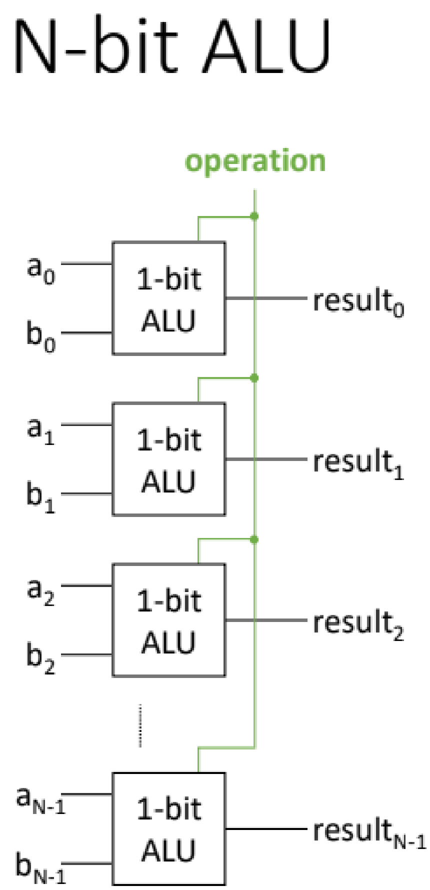
**Example: 1-bit ALU**

A 1-bit ALU performs:

- AND
- OR
- ADD
- SUBTRACT

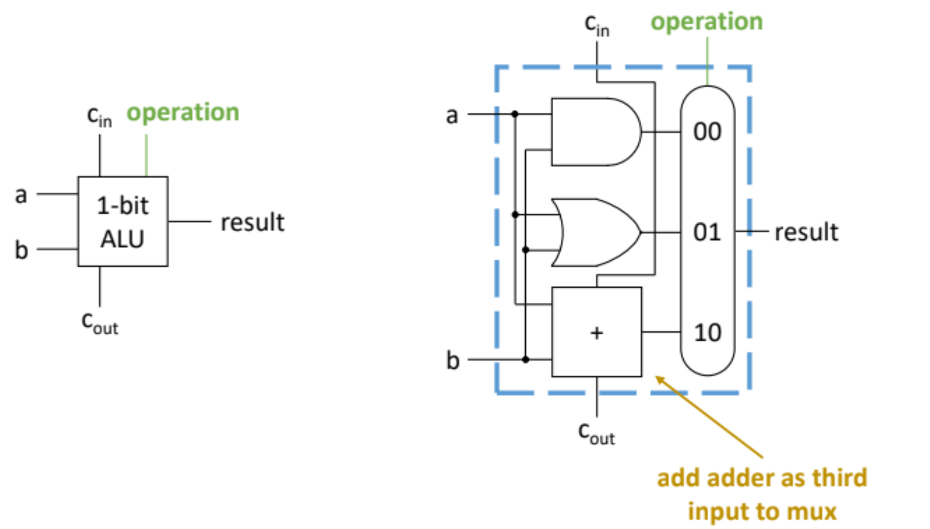


**Example: N-bit ALU**

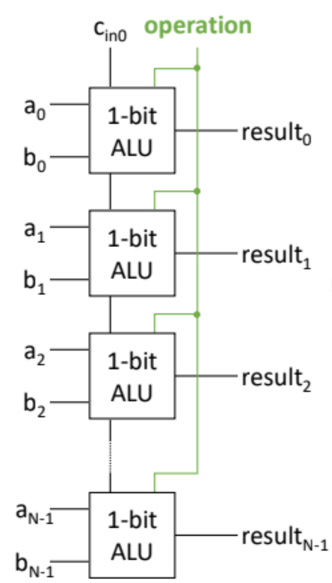


**Example: Addition**

# 1-bit ALU with Addition

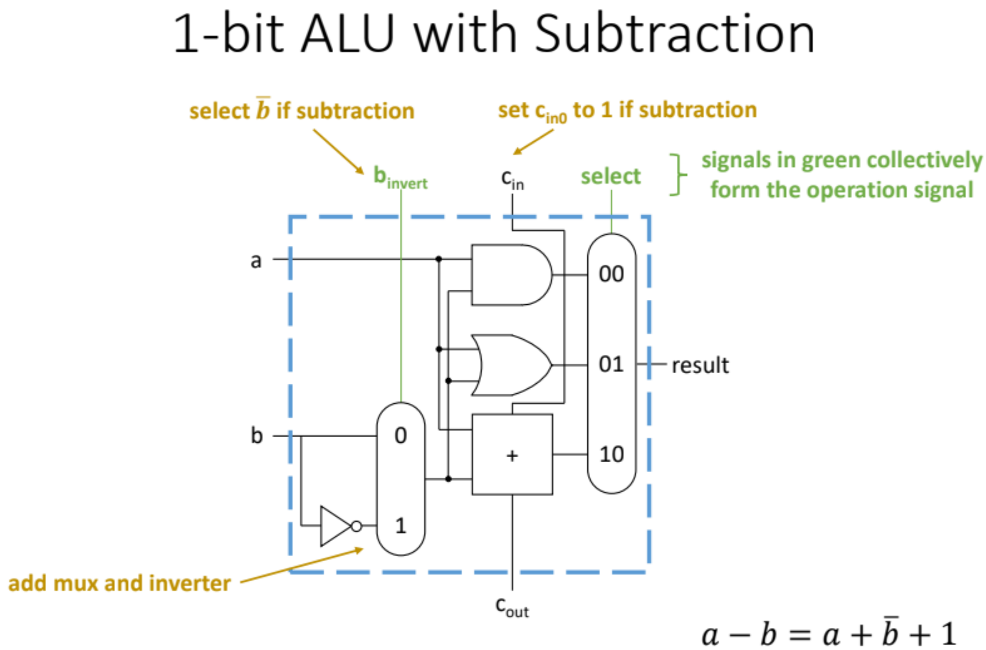



# N-bit ALU with Addition

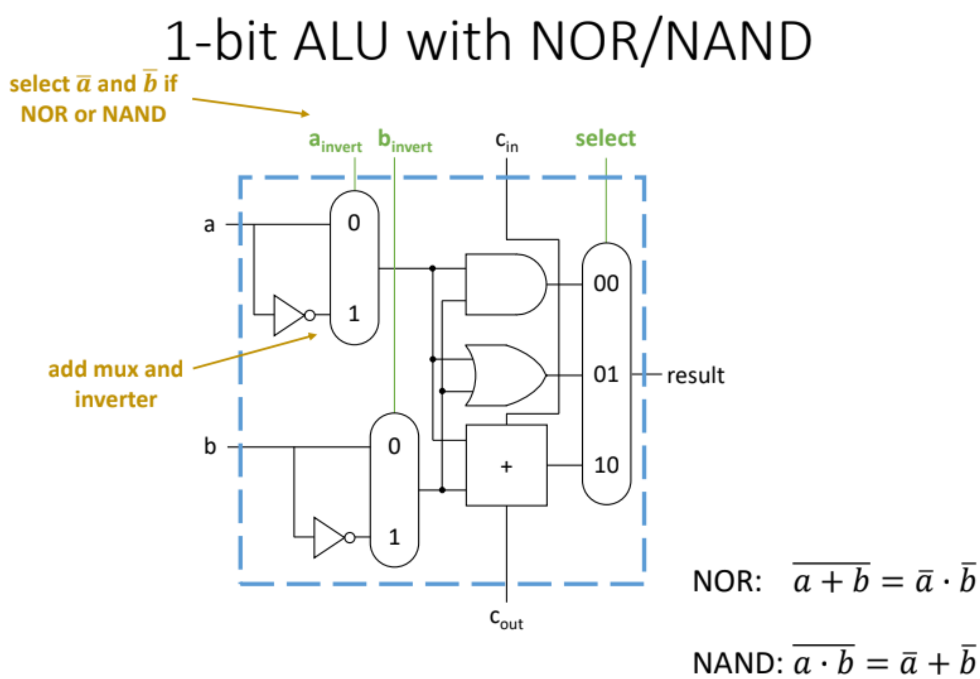



The adder created by this configuration is called a **ripple carry adder** because the carry "ripples" across the 1-bit adder

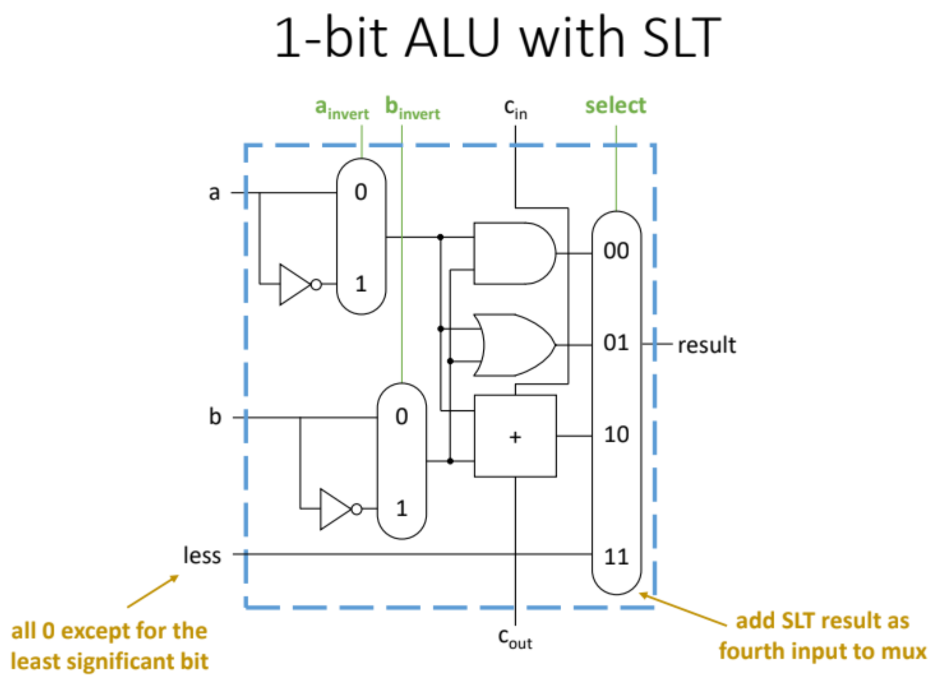
**Example: Subtraction**

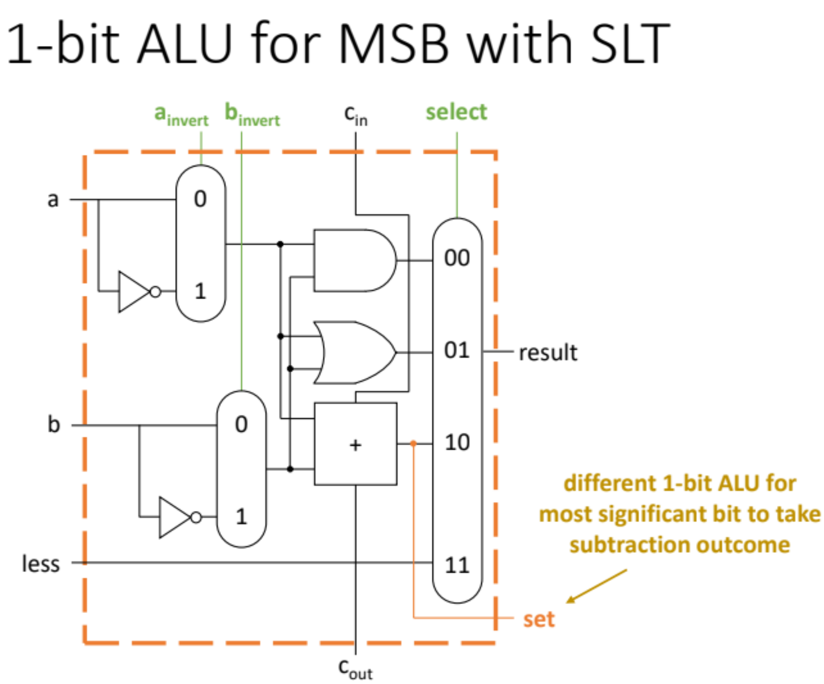


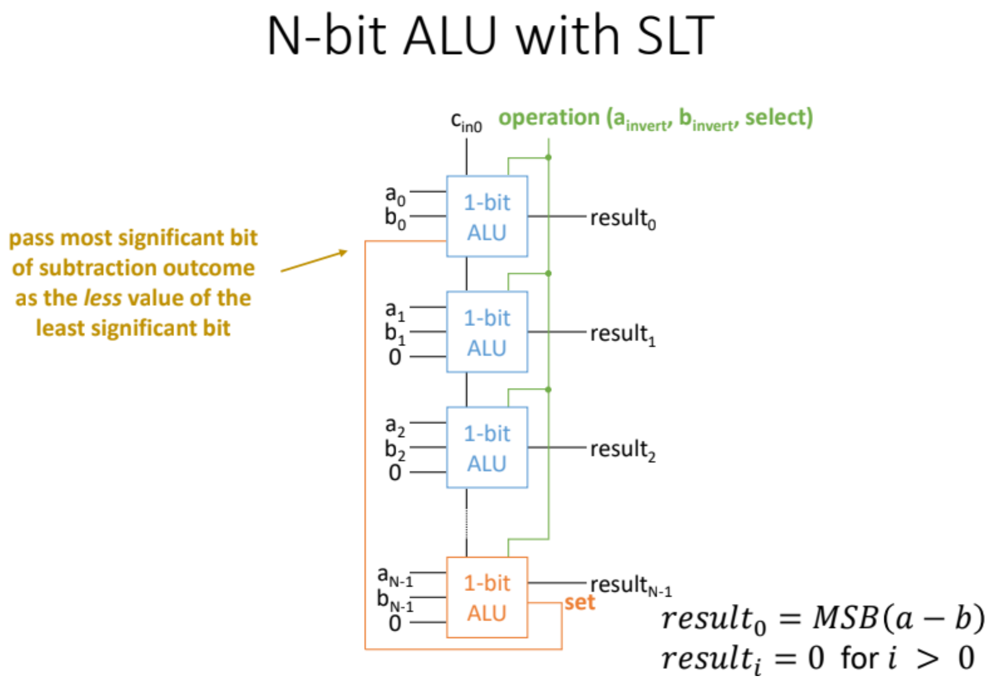

**Example: NOR and NAND**



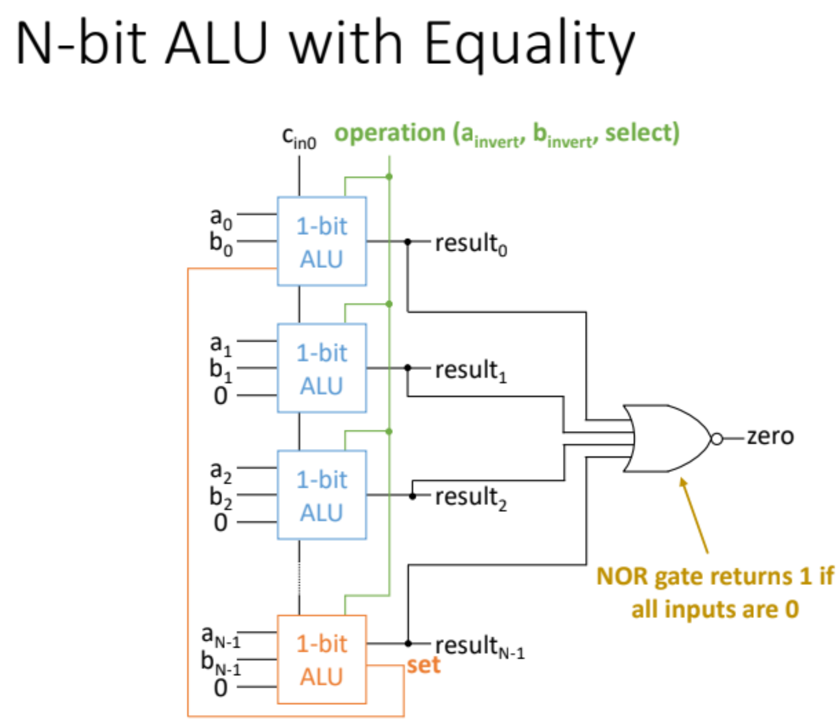
**Example: Set on Less Than**



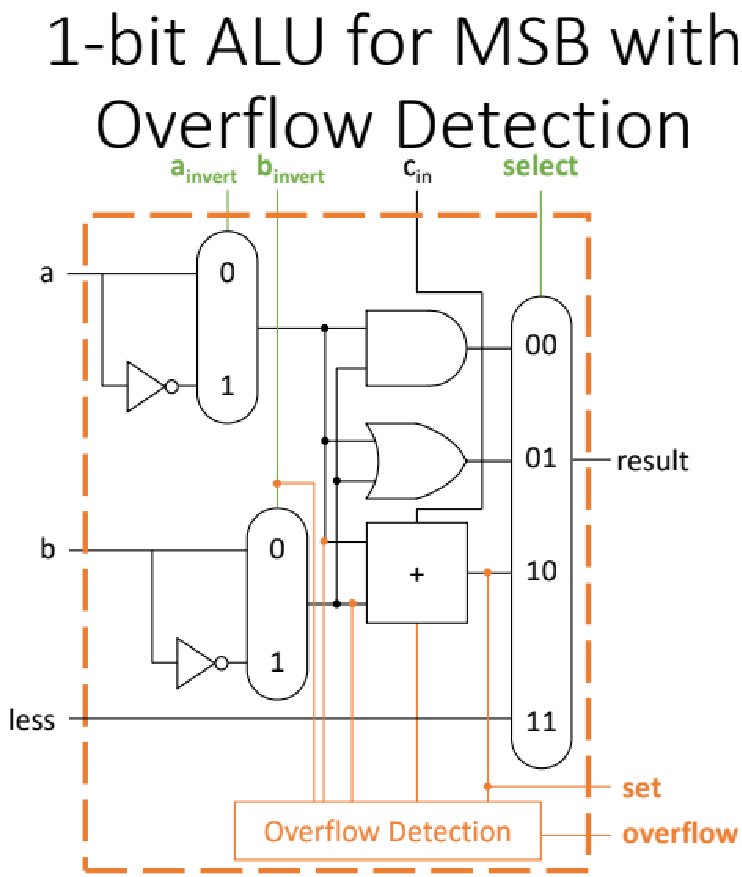



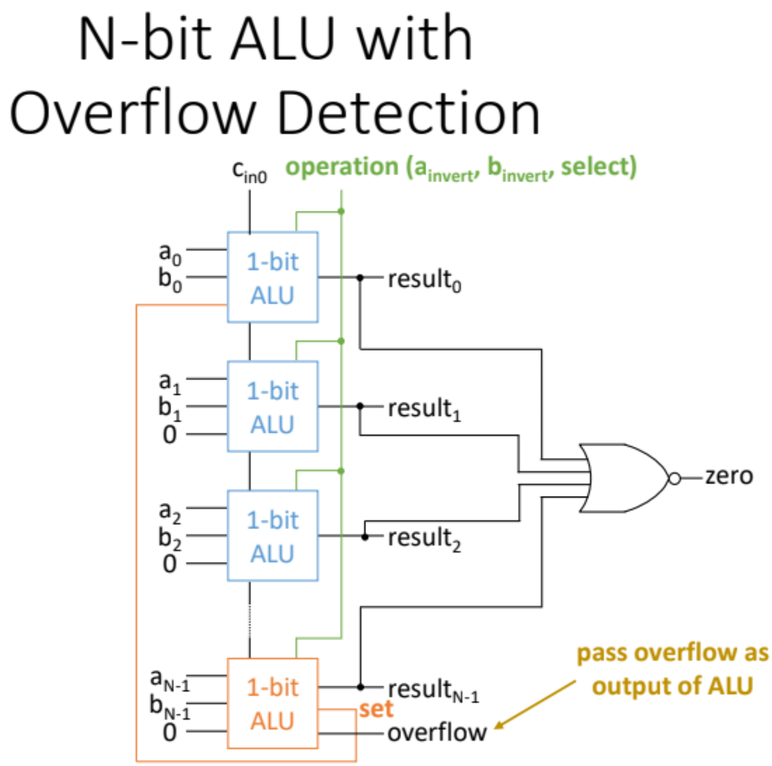



**Example: Equality**



**Example: Overflow Detection**






## 4 Status Flags

The ALU sets several status flags after an operation:

- **Zero (Z):** Result is zero
- **Carry (C):** Carry out of MSB in addition
- **Overflow (V):** Signed overflow occurred
- **Negative (N):** Result is negative (MSB = 1)

| Flag | Description |
|---|---|
| Z | Zero result |
| C | Carry out |
| V | Signed overflow |
| N | Negative result |

## 5 ALU Control Logic

The ALU control unit uses an opcode and function bits to select the correct operation.

| Instruction opcode | ALUOp | Instruction operation | Funct field | Desired ALU action | ALU control input |
|---|---|---|---|---|---|
| LW | 00 | load word | XXXXXX | add | 0010 |
| SW | 00 | store word | XXXXXX | add | 0010 |
| Branch equal | 01 | branch equal | XXXXXX | subtract | 0110 |
| R-type | 10 | add | 100000 | add | 0010 |
| R-type | 10 | subtract | 100010 | subtract | 0110 |
| R-type | 10 | AND | 100100 | and | 0000 |
| R-type | 10 | OR | 100101 | or | 0001 |
| R-type | 10 | set on less than | 101010 | set on less than | 0111 |

## 6 Conclusion

The ALU is an essential building block of any computing system. Its ability to perform a variety of operations makes it the computational brain of the processor. Understanding ALU design and function is critical for comprehending processor architecture and digital logic design.